GUIA DE IMPLANTAÇÃO

DNAutos Vistorias

Guia de integração, segurança e hospedagem Preparado em 11/11/2026 • Aplicação React + Express + Supabase

1. O que está pronto e o que falta

O fluxo implementado é: cadastro por e-mail e senha, confirmação de e-mail, login, segundo fator, página inicial, escolha da placa, compra de R$ 50,00, confirmação do pagamento pelo servidor e consulta pelo histórico de compras.

Frontend e backend usam o mesmo domínio e o prefixo /api. O Supabase hospeda Auth e PostgreSQL. Os tokens do Supabase permanecem criptografados no backend; o navegador recebe um identificador aleatório em cookie HttpOnly, Secure em produção e SameSite=Lax. O servidor expira a sessão após 30 minutos sem atividade e exige novo login depois de 24 horas, mesmo com atividade.

A senha não é salva nas tabelas da aplicação. O Supabase Auth armazena hash bcrypt com salt próprio. E-mail é o único dado cadastral obrigatório; o celular é solicitado para o segundo fator. Google não substitui a exigência de MFA.

Uma compra pertence a um cliente e uma placa imutável. O preço está fixado em 5000 centavos no servidor e por CHECK no banco. Alterar HTML, JavaScript, URL de retorno ou status enviado pelo navegador não libera relatório.

As vendas ficam desativadas por padrão. Faltam criar o Supabase, configurar SMTP, Google e SMS, contratar a ANYCAR, validar seu contrato e configurar o Mercado Pago. Não houve conexão a contas reais nem deploy nesta entrega.

A integração ANYCAR é um adaptador provisório, herdado e isolado. Não é possível garantir que bastará inserir uma chave: método, URL, cabeçalho, produto e resposta precisam coincidir com a documentação contratada. Existe uma trava explícita antes de vender.

Fonte sobre armazenamento de senhas: https://supabase.com/docs/guides/auth/password-security

2. Preparar a máquina e executar localmente

1. Instale Node.js 24 LTS, Git e um editor. Use uma cópia confiável deste repositório. Mantenha os dois package-lock.json versionados.

2. Abra o terminal na raiz e instale as dependências:

```
npm ci
npm ci --prefix backend
```

3. No PowerShell, crie os arquivos locais de configuração:

```
Copy-Item .env.example .env
Copy-Item backend/.env.example backend/.env
```

4. Deixe VITE_API_URL=/api. Em backend/.env, mantenha PUBLIC_APP_URL=http://localhost:5110 e PUBLIC_API_URL=http://localhost:3000. Use sempre o mesmo nome de host no navegador.

5. Gere três segredos independentes executando o comando abaixo três vezes. Salve cada saída de forma privada em SESSION_ENCRYPTION_KEY, AUTH_RATE_SECRET e CRON_SECRET:

```
node -e "console.log(require('crypto').randomBytes(32).toString('base64'))"
```

6. Configure o Supabase nas próximas páginas e execute:

```
npm run dev:all
```

7. Acesse http://localhost:5110. A API responde em /api/health. Sem configuração, as funções protegidas retornam indisponibilidade e nenhum pagamento real é criado.

Para testar pagamentos e callbacks, use um ambiente de homologação HTTPS com contas de teste, origem fixa e URLs correspondentes. Não use credenciais reais em um túnel compartilhado.

Nunca envie .env para Git, arquivos ZIP de publicação, prints, tickets ou o frontend. As variáveis VITE_* são públicas. DATABASE_URL, SESSION_ENCRYPTION_KEY e credenciais de gateway/provedor são exclusivas do servidor.

3. Criar o projeto Supabase e o esquema

1. Crie um projeto Supabase. Escolha a região considerando seus clientes e a hospedagem da API. Defina senha forte para o administrador do banco.

2. Guarde a referência do projeto, SUPABASE_URL e a chave publishable. Não é necessário usar service_role neste backend.

3. No SQL Editor do projeto novo, execute integralmente o arquivo:

```
supabase/migrations/
20261111173427_secure_vehicle_platform.sql
```

4. Execute apenas uma vez. O arquivo foi criado com a CLI de migrations e deve permanecer versionado. Em uma base com dados existentes, revise e planeje uma migração específica antes de executar alterações.

5. Confira o schema benz. Ele contém profiles, sessions, login_guards, orders, reports, webhook_jobs, support_requests e rate_limits.

6. Confirme RLS nas oito tabelas. Não adicione benz aos schemas expostos pelo Data API. Não conceda acesso a anon ou authenticated. Não crie políticas públicas para “fazer funcionar”.

7. O vínculo de profiles.id com auth.users.id preserva a identidade do Supabase. O perfil é criado no primeiro login válido; compras e relatórios usam essa mesma identidade.

8. Configure backups no plano contratado e faça um teste de restauração separado. Confirme a retenção e a disponibilidade de PITR no plano atual.

Não existem senhas de clientes em profiles. Telefones e fatores ficam no Supabase Auth. O histórico não é salvo em localStorage. A aplicação remove os antigos dados locais ao iniciar.

Este esquema foi validado em PostgreSQL embarcado nos testes. Ainda é necessário executar a migration e os advisors no seu Supabase real e confirmar privilégios, extensões e conexão.

Fonte de segurança do banco: https://supabase.com/docs/guides/database/postgres/row-level-security

4. Conexão ao banco com privilégio restrito

1. Depois da migration, execute supabase/runtime-role.sql como administrador. Ele cria o grupo benz_backend sem login, concede acesso apenas ao schema benz e adiciona políticas para esse grupo de servidor.

2. Crie um login separado e conceda o grupo. Substitua o exemplo de senha diretamente no ambiente privado, nunca no repositório:

```
CREATE ROLE benz_app LOGIN INHERIT
PASSWORD 'SUBSTITUA_POR_UMA_SENHA_FORTE';
GRANT benz_backend TO benz_app;
```

3. Em Connect no Supabase, copie os dados do pooler. Para Vercel/serverless, use transaction mode, normalmente porta 6543. Para processo persistente em VPS, o session pooler, normalmente 5432, também é adequado.

4. Use o login benz_app na conexão. Em Supavisor o nome costuma ser benz_app.PROJECT_REF; confirme o formato no projeto e no pooler escolhido. Codifique caracteres especiais da senha na URL.

5. Preencha DATABASE_URL no servidor. Não use a conta postgres como credencial permanente da aplicação.

6. Baixe o certificado CA correspondente no painel. Em produção, o cliente exige validação TLS; configure DATABASE_CA_CERT se a CA não estiver no trust store. Uma variável multilinha pode usar a sequência literal \n.

7. Não desative rejectUnauthorized para contornar problemas. Verifique a CA, host, porta, usuário, senha e rede.

8. Execute:

```
npm --prefix backend run doctor
```

O diagnóstico imprime apenas valores booleanos, verifica tabelas/RLS e identifica uma conexão com superusuário ou BYPASSRLS. Antes de contratar/configurar tudo, é esperado que algumas verificações falhem.

Não conceda a benz_app acesso a auth.users ou auth.sessions. A autenticação ocorre pela API Auth do Supabase; o banco da aplicação fica separado.

Fonte: https://supabase.com/docs/guides/database/connecting-to-postgres

5. Configurar e-mail, confirmação e recuperação

1. Em Authentication, habilite e-mail/senha e mantenha a confirmação de e-mail obrigatória. Desative cadastro anônimo se não for utilizado.

2. Defina senha mínima de 12 caracteres e configure proteção contra senhas vazadas no plano que ofereça o recurso. O backend limita a entrada a 72 bytes para não truncar senhas bcrypt silenciosamente.

3. Configure um provedor SMTP de produção. Valide o domínio de envio e seus registros SPF/DKIM/DMARC com o provedor. Teste entrega e pasta de spam.

4. Em URL Configuration, informe Site URL=https://consulta.seudominio.com.br. Cadastre também o ambiente local ou de homologação, separadamente.

5. Personalize o template Confirm sign up com este link. O fragmento após # não é enviado como URL para o servidor:

```
<a href="{{ .SiteURL }}/#token_hash={{ .TokenHash }}&amp;type=signup">
  Confirmar meu e-mail
</a>
```

6. No template Reset password, use:

```
<a href="{{ .SiteURL }}/#token_hash={{ .TokenHash }}&amp;type=recovery">
  Redefinir senha e liberar acesso
</a>
```

7. Desative rastreamento de cliques nos links de autenticação. O app limpa o fragmento da barra de endereço e só consome o link quando o usuário confirma a ação.

8. Teste cadastro, reenvio, link expirado e recuperação. Após cinco senhas incorretas, a aplicação bloqueia o acesso. O link de recuperação cria uma sessão restrita; a liberação ocorre após definir nova senha. Se já houver MFA, a recuperação preserva essa proteção.

9. Habilite notificações de segurança relevantes no Supabase, como alteração de senha ou fator.

Não use o template padrão que redireciona tokens diretamente ao navegador: esta implementação espera token_hash e os dois tipos acima. Novos projetos com SMTP padrão podem ter limitações de personalização; configure SMTP próprio.

Fontes: https://supabase.com/docs/guides/auth/auth-email-templates
https://supabase.com/changelog/46599-changes-to-email-template-customisation-on-free-tier

6. Google e código do celular

Google:

1. No Google Cloud, configure o projeto, a tela de consentimento e um cliente OAuth do tipo aplicação Web.

2. Cadastre como URI de redirecionamento do Google o callback mostrado pelo Supabase, normalmente https://PROJECT_REF.supabase.co/auth/v1/callback.

3. No Supabase, habilite o provedor Google e informe Client ID e Client Secret. Não coloque o segredo no frontend.

4. Na lista de redirecionamentos permitidos do Supabase, cadastre https://consulta.seudominio.com.br/api/auth/callback**. O sufixo permite o parâmetro flow gerado pelo servidor; mantenha o domínio exato.

5. O backend usa PKCE, cookie temporário criptografado e expiração de dez minutos. Não aceite redirects arbitrários. Teste também cancelamento, erro e fluxo iniciado em outro navegador.

SMS/MFA:

1. Configure um provedor de SMS compatível no Supabase, com conta ativa, remetente e autorização para os países atendidos.

2. Habilite enrollment e verification de Phone MFA e códigos com pelo menos seis dígitos. Configure limites e orçamento de envio.

3. Mantenha REQUIRE_PHONE_MFA=true. O primeiro login pede telefone com DDI; contas já configuradas precisam provar o fator em cada nova sessão.

4. Teste número inválido, código incorreto, expiração e reenvio. Código e fator são vinculados à sessão do servidor. Nenhuma rota de compra/relatório aceita uma sessão sem a segurança exigida.

5. Defina uma operação de suporte para celular perdido. Não remova um fator apenas por uma mensagem de WhatsApp. A versão atual não possui tela de troca de telefone nem recuperação administrativa automatizada.

O custo de SMS e eventual habilitação/cobrança de Phone MFA devem ser confirmados no plano Supabase atual. O acesso não contorna MFA quando o SMS está indisponível.

Fontes: https://supabase.com/docs/guides/auth/social-sign-in/auth-google https://supabase.com/docs/guides/auth/auth-mfa/phone

7. Conectar o Mercado Pago

1. Crie a aplicação no painel de desenvolvedores e escolha o fluxo Checkout Pro com Preferences + Payments API. Este código usa /checkout/preferences e /v1/payments; não misture com uma integração Orders diferente sem adaptar o código.

2. Obtenha o Access Token, o identificador da conta recebedora e o segredo da assinatura do webhook. Preencha MERCADO_PAGO_ACCESS_TOKEN, MERCADO_PAGO_COLLECTOR_ID e MERCADO_PAGO_WEBHOOK_SECRET.

3. Em testes, use MERCADO_PAGO_LIVE_MODE=false e as contas/credenciais de teste indicadas pelo gateway. O link usa sandbox_init_point nesse modo.

4. Cadastre a URL HTTPS do webhook:

```
https://consulta.seudominio.com.br/api/checkout/webhook
```

5. Ative eventos de pagamentos compatíveis com /v1/payments. O endpoint espera data.id na query, x-signature e x-request-id. Verifica HMAC-SHA256 com tolerância de cinco minutos; mantenha o relógio do host sincronizado.

6. O backend busca o pagamento autenticado no gateway e confere valor, BRL, conta recebedora, referência da compra e ambiente. Conteúdo do body e parâmetros de retorno não provam pagamento.

7. A preferência é exclusiva da compra. O Checkout Pro gera o Pix/QR Code correspondente no ambiente do gateway. Não é um QR estático nem a API presencial de QR, que tem regras diferentes de assinatura.

8. Pix, crédito e débito dependem dos meios habilitados e disponibilizados pelo Mercado Pago para a conta e a transação. A aplicação não coleta número de cartão ou CVV. Confira débito na homologação com o gateway.

9. Em produção, use a credencial correspondente, MERCADO_PAGO_LIVE_MODE=true e o segredo de produção. Faça uma compra controlada, concilie e estorne pelo painel.

O recebimento válido gera um registro durável e é processado durante a chamada. Falhas deixam trabalho para repetição do gateway e manutenção. Eventos duplicados não criam créditos extras. Estorno e contestação suspendem acesso futuro ao relatório.

Fontes: https://www.mercadopago.com.br/developers/pt/docs/subscriptions/additional-content/your-integrations/notifications/webhooks
https://github.com/mercadopago/sdk-nodejs/blob/master/docs/webhooks.md

8. Conectar a ANYCAR com segurança

1. Solicite ao fornecedor a documentação do produto contratado: host, método, autenticação, tipos de consulta, exemplos reais anonimizados, erros, limites, timeout, cobrança e política de repetição.

2. Confirme o custo total. Uma compra de R$ 50,00 pode chamar mais de um tipo em ANYCAR_REPORT_TYPES; isso pode consumir vários créditos do fornecedor.

3. O adaptador atual pressupõe POST {ANYCAR_API_URL}/api/consultar, cabeçalho x-api-key, body { tipo, placa } e resposta { status, dados }. Essa forma ainda NÃO foi validada contra seu contrato.

4. Compare o contrato com backend/src/services/anycarService.ts e ajuste transporte e toReport quando necessário. Configure ANYCAR_API_URL, ANYCAR_API_KEY e ANYCAR_REPORT_TYPES apenas para os produtos autorizados.

5. Use fixtures anonimizadas do contrato para verificar os cartões de situação cadastral, finanças, leilão, sinistro, restrições, proprietário/quilometragem e FIPE quando disponíveis. Um dado ausente deve aparecer como não informado.

6. Não infira IPVA regular, origem Detran, gravidade de sinistro, score ou ausência de ocorrência a partir de campos faltantes. Mapeamentos dependentes do contrato foram deixados conservadores.

7. A resposta bruta do fornecedor não é exibida nem persistida automaticamente. Revise cada campo permitido, principalmente dados pessoais de proprietários, e a necessidade de armazená-lo.

8. Valide o fornecedor em homologação com uma placa autorizada. Se não existir sandbox, combine previamente uma consulta controlada e seu custo.

9. Só depois da conferência técnica configure ANYCAR_CONTRACT_VERIFIED=true. Ative CHECKOUT_ENABLED=true quando TODO o fluxo estiver homologado.

O servidor reserva uma compra antes de chamar o fornecedor. Uma consulta bem-sucedida é salva e reutilizada. Em timeout, resultado incerto ou queda do processo, o pedido vai para processamento/análise e não dispara outra consulta automaticamente. O operador deve conciliar com o fornecedor antes de liberar uma tentativa ou devolver o pagamento.

9. Variáveis e diagnóstico antes da publicação

Use backend/.env.example como inventário. Estas são as categorias:

- URLs públicas: PUBLIC_APP_URL e PUBLIC_API_URL. Em produção, configure ambas com o mesmo domínio HTTPS, sem /api e sem barra final.

- Identidade: SUPABASE_URL, SUPABASE_PUBLISHABLE_KEY, REQUIRE_PHONE_MFA=true.

- Persistência: DATABASE_URL do login restrito e DATABASE_CA_CERT se necessária para validar TLS.

- Sessão: SESSION_ENCRYPTION_KEY de 32 bytes em base64 e AUTH_RATE_SECRET independente.

- Pagamentos: token, segredo do webhook, collector ID e MERCADO_PAGO_LIVE_MODE.

- Consulta: URL/chave/produtos ANYCAR, ANYCAR_CONTRACT_VERIFIED e CHECKOUT_ENABLED.

- Operação: SUPPORT_URL com URL HTTPS real de atendimento e CRON_SECRET aleatório.

- Hospedagem: NODE_ENV=production; TRUST_PROXY=1 somente atrás de um proxy confiável; SERVE_FRONTEND=true em VPS/processo único.

- FIPE: FIPE_API_URL e FIPE_API_KEY quando o fornecedor exigir.

Não existe REPORT_PRICE configurável pelo cliente. Para alterar o preço futuramente, será necessário revisar código do servidor, constraint do banco, interface, testes e política comercial em uma mudança coordenada.

O endpoint público /api/config só informa disponibilidade, preço e contato. Não retorna credenciais. /api/health confirma que o processo responde; isso não prova que os fornecedores ou o banco estão saudáveis.

Antes de abrir vendas:

```
npm run build:all
npm test
npm --prefix backend run doctor
```

Execute também os testes de navegador. No Windows com Edge instalado:

```
npx playwright test
```

Em outra plataforma, configure Playwright com um navegador disponível. Não publique segredos como artefatos de testes.

10. Publicar temporariamente na Vercel

1. Versione a implementação em um repositório privado ou com os segredos devidamente excluídos. Confirme que .env não aparece em git status.

2. Importe a raiz do repositório para um projeto Vercel. O vercel.json existente usa Services: app para Vite e backend para Express.

3. Mantenha a raiz do projeto no repositório, o Node.js em 24.x e a configuração de Services. Não transforme o backend em um site estático.

4. O frontend atende /. O backend recebe /api/* no mesmo domínio. VITE_API_URL=/api mantém cookies e evita configuração de autenticação entre domínios.

5. Cadastre as variáveis do backend no ambiente correto do projeto. Credenciais, URL do banco e chaves de sessão devem ser tratadas como Secret. Preview deve usar serviços/contas de teste.

6. Configure PUBLIC_APP_URL e PUBLIC_API_URL com o domínio público estável. Ajuste callbacks no Supabase e webhook no Mercado Pago antes de habilitar vendas.

7. Faça um deploy de homologação. Confira /api/health, /api/config, assets e uma URL interna. O build deve incluir ambos os serviços.

8. O backend solicita duração máxima de 120 segundos. Confirme o limite do plano e as condições comerciais atuais antes de usar pagamentos reais. Não dependa de disco local nem de tarefas após enviar a resposta.

9. Programe uma chamada autenticada de manutenção. Em Vercel, use um agendador compatível com o plano; em outro scheduler, envie Authorization: Bearer CRON_SECRET para /api/maintenance/reconcile a cada cinco minutos. Não exponha o segredo em query.

10. Proteção de deployment não pode impedir a entrega do webhook em produção. Preserve a autenticação interna do app e a validação da assinatura; não abra ambientes privados com dados reais indiscriminadamente.

Uma alteração em variáveis exige novo deploy para o ambiente correto. Após homologar, promova a versão testada. Nenhum deploy foi executado automaticamente nesta tarefa.

Fontes: https://vercel.com/docs/services https://vercel.com/docs/frameworks/backend/express https://vercel.com/docs/environment-variables

11. Hostinger: frontend e backend juntos

Sim, é possível manter este frontend e o backend juntos. O requisito é um plano que execute Node.js persistente e permita conexões de saída HTTPS/PostgreSQL. O banco e o Auth continuam no Supabase.

A Hostinger oferece opções de aplicação Node.js em planos elegíveis, além de VPS. Confira a disponibilidade exata no plano e painel antes de contratar. Hospedar apenas dist em uma hospedagem estática não executa a API.

Para um plano gerenciado de aplicação Node.js:

1. Confirme Node.js 24, customização de build/start, variáveis de ambiente, HTTPS e saída para o pooler do Supabase.

2. Importe o repositório completo a partir da raiz. Inclua backend; publicar somente src ou dist não basta.

3. Configure como build:

```
npm ci
npm ci --prefix backend
npm run build:all
```

4. Use como comando de inicialização:

```
node backend/dist/server.js
```

5. Configure NODE_ENV=production e SERVE_FRONTEND=true. Use a porta fornecida pela hospedagem via PORT. Ative TRUST_PROXY=1 apenas se a plataforma realmente colocar um proxy confiável à frente.

6. Cadastre as mesmas variáveis privadas, com as URLs finais. O backend serve dist e /api no mesmo processo.

7. Ligue o domínio, habilite HTTPS e teste login, compra, webhook e histórico. Programe a manutenção por um agendador externo se o plano não oferecer cron.

8. Se o painel não permitir este monorepo, o build ou o processo persistente, use a opção de VPS com Docker descrita na próxima página.

Fonte: https://www.hostinger.com/support/node-js-hosting-options-at-hostinger/ Tutorial do provedor: https://www.hostinger.com/support/how-to-deploy-a-nodejs-website-in-hostinger/

12. VPS Hostinger ou HostGator com Docker

A VPS permite executar a aplicação e o proxy no mesmo servidor, mantendo Supabase externo. Há suporte a Docker/Node.js em opções de VPS dos provedores; confirme recursos e responsabilidades de administração. Não presuma suporte Node.js em um plano compartilhado comum.

1. Provisione uma VPS nova ou um ambiente sem conflito com outro proxy nas portas 80/443. Escolha sistema operacional suportado pelo Docker. Atualize o sistema e siga a instalação oficial de Docker Engine e Compose para essa distribuição.

2. Configure acesso SSH por chave, usuário administrativo separado e firewall do provedor. Libere 80/443 e restrinja SSH. Não publique PostgreSQL nem a porta 3000.

3. Aponte o registro DNS do subdomínio para a VPS. Remova um registro AAAA incorreto se a VPS não responder por IPv6.

4. Clone o repositório em uma pasta de aplicação. Crie backend/.env com as credenciais finais. Crie deploy/.env a partir de deploy/.env.example.

5. Em deploy/.env, defina APP_DOMAIN=consulta.seudominio.com.br e ACME_EMAIL com e-mail administrativo real. Não use protocolo em APP_DOMAIN.

6. No Linux, restrinja os arquivos de segredos:

```
chmod 600 backend/.env deploy/.env
```

7. Na raiz do projeto, valide e inicie:

```
docker compose --env-file deploy/.env -f deploy/compose.yaml config --quiet
docker compose --env-file deploy/.env -f deploy/compose.yaml up -d --build
```

8. O container Node roda sem root e serve frontend+API. O Caddy encaminha as requisições e obtém o certificado quando DNS e portas estão corretos. Seus volumes preservam os certificados.

9. Confira saúde e logs:

```
docker compose --env-file deploy/.env -f deploy/compose.yaml ps
docker compose --env-file deploy/.env -f deploy/compose.yaml logs --tail=100 app
```

10. Agende a manutenção, monitore disco/memória e atualize imagens. Valide recuperação do Supabase e dos volumes de TLS. Mantenha uma versão anterior para rollback.

O Dockerfile/Compose foram preparados, mas não foram executados nesta máquina por falta de Docker. Valide o build na homologação antes de direcionar clientes.

Fontes: https://docs.docker.com/engine/install/ https://caddyserver.com/docs/quick-starts/reverse-proxy https://suporte.hostgator.com.br/hc/pt-br/articles/45908909421331-Como-adicionar-um-painel-ou-aplica%C3%A7%C3%A3o-%C3%A0-sua-VPS-HostGator

13. Migrar de hospedagem sem perder compras

1. Mantenha o mesmo projeto Supabase e os dados. A aplicação não depende de SQLite ou arquivos de relatório no host.

2. Reduza o TTL DNS com antecedência. Prepare a nova hospedagem com o mesmo código e variáveis, inicialmente com CHECKOUT_ENABLED=false.

3. Se usar o mesmo domínio final, mantenha as URLs públicas. Se trocar o domínio, altere Site URL, templates, callbacks Google/Supabase e webhook Mercado Pago para os endereços novos.

4. Confirme que o certificado HTTPS e as rotas /api estão funcionando no host novo. Use uma homologação separada para testar sem desviar notificações reais.

5. Pause novas vendas na origem durante a virada. Não desligue o webhook antigo enquanto ainda recebe eventos de compras anteriores.

6. Use o mesmo banco e contrato de dados nos dois hosts durante a transição. Não execute migrations destrutivas durante a troca.

7. Aponte DNS e atualize o webhook. Verifique entregas nos dois ambientes e os pedidos pendentes. Quando a nova origem estiver validada, reative vendas.

8. Confira credenciais e flags: teste usa MERCADO_PAGO_LIVE_MODE=false; produção usa true com a conta recebedora correta.

9. Preserve SESSION_ENCRYPTION_KEY e AUTH_RATE_SECRET se quiser manter compatibilidade das sessões e bloqueios durante a mudança. Trocar essas chaves exige um plano: revogar sessões antigas e tratar login_guards sem desativar bloqueios por acidente.

10. Mantenha o host anterior disponível durante a propagação. Em falha, pause vendas, reverta DNS/versão e reconcilie os pagamentos já recebidos.

Cookies pertencem ao domínio. Ao trocar de domínio, o usuário precisará entrar novamente; as compras continuam no banco. Uma migração não deve criar um novo projeto Supabase vazio.

Nunca considere a página “pagamento aprovado” como confirmação. O histórico deve refletir o pagamento conferido no servidor, inclusive se o cliente fechar a aba do gateway.

14. Atendimento, estorno e manutenção

O botão Ajuda e estorno usa SUPPORT_URL, por exemplo um endereço HTTPS do WhatsApp da empresa ou um portal de suporte. As vendas exigem um canal configurado. No histórico, o cliente pode abrir uma solicitação e recebe um protocolo persistido em support_requests.

A solicitação não executa estorno automaticamente e não promete prazo de devolução. O operador deve conferir identidade, compra, transação e consumo do relatório, e efetuar a devolução no painel do Mercado Pago conforme a política aplicável. O webhook recebido depois atualiza o acesso. Não libere consultas com um comprovante enviado pelo cliente.

Operação diária:

- Consulte solicitações abertas em benz.support_requests pelo ambiente administrativo restrito. Não existe painel de administrador nesta versão.

- Acompanhe webhook_jobs com last_error ou attempts elevados. Divergências são bloqueadas para análise; indisponibilidade do gateway pode ser repetida.

- Compras pagas com report_state=review exigem conciliação com a ANYCAR. Não altere para available sem provar que não houve consumo ou obter confirmação do fornecedor.

- Uma solicitação de estorno pausa uma consulta ainda não realizada. Encerrar o ticket, quando adequado, é operação administrativa autenticada no banco.

- A chamada GET /api/maintenance/reconcile exige Authorization: Bearer CRON_SECRET. Ela reprocessa até cinco notificações pendentes, marca processamentos antigos para análise e remove sessões/rate limits antigos.

- Após 20 tentativas, uma notificação requer intervenção. O endpoint não descobre todos os pagamentos que nunca geraram webhook: confira também o painel do gateway e reenvie a notificação autenticada.

Monitore erros sem registrar senhas, tokens, assinaturas completas, placas em URLs de logs ou respostas brutas de fornecedores. O app gera X-Request-ID para diagnóstico. Configure também os logs do proxy e serviços externos para não guardar links de autenticação.

Defina retenção de dados, exclusão e atendimento à privacidade com responsáveis pelo negócio. Backups e registros financeiros podem exigir tratamento diferente do histórico exibido ao cliente.

15. Homologação obrigatória antes de vender

Execute os comandos de build/teste e registre o resultado. Os testes locais usam PostgreSQL embarcado e respostas simuladas do Supabase/Mercado Pago/ANYCAR; eles validam a lógica do app, não substituem testes com contas reais.

Autenticação:

- Cadastro sem nome, confirmação/reenvio e link inválido.

- Google com callback correto e callback sem cookie PKCE.

- SMS correto, incorreto, vencido, reenvio e orçamento.

- Cinco senhas incorretas, bloqueio e desbloqueio por recuperação.

- Logout, expiração de 30 minutos sem interação e uso em duas abas.

- Tentativa de acessar outro cliente e chamadas sem cookie/MFA.

Pagamento:

- Pix e cartão com comprador de teste diferente do vendedor.

- Valor adulterado, moeda diferente, collector incorreto e ambiente errado.

- Webhook ausente, inválido, repetido, fora da tolerância e entregas atrasadas.

- Cliente fecha a aba antes do retorno; compra ainda aparece no histórico.

- Pagamento recusado, estorno parcial/total, contestação e novo evento antigo.

- Duplo clique e repetição do mesmo Idempotency-Key.

Consulta:

- Compra pendente nunca chama a ANYCAR.

- Placa fixa da compra, único resultado persistido e releitura sem nova chamada.

- Timeout do provedor, falha do banco e processo interrompido: compra preservada e análise manual.

- Campos desconhecidos aparecem como não informados.

- Sem exposição automática de resposta bruta/dados pessoais desnecessários.

Infraestrutura:

- TLS verificado no banco e no domínio; cookie Secure em produção.

- Login restrito no PostgreSQL, RLS, advisors e backups restauráveis.

- Webhook acessível, cron autenticado, suporte funcional e rollback ensaiado.

- npm audit nos dois pacotes e revisão de dependências antes de cada release.

Esta base reduz riscos conhecidos. Não é possível prometer ausência de vulnerabilidades ou falhas de terceiros; mantenha atualizações, monitoramento e revisão periódica.

16. Limites da entrega e próximos passos

Implementado no repositório: fronteiras de autenticação/MFA, confirmação e recuperação, sessões de servidor, compras por placa, preço fixo, validação de webhook/pagamento, histórico, solicitação de suporte/estorno, persistência, migrations, RLS, isolamento de serviços e arquivos de hospedagem.

Verificado localmente: builds TypeScript/Vite, testes automatizados de segurança/negócio e cenários de navegador. Os detalhes e comandos ficam em docs/VERIFICACAO.md.

Não executado: criação de projeto Supabase, envio real de e-mail/SMS, aprovação OAuth real, pagamento Pix/cartão real, assinatura de webhook real, consulta contratada ANYCAR, build Docker e deploy público. Essas atividades dependem de serviços/credenciais ainda inexistentes e de homologação na infraestrutura escolhida.

Pendências conscientes: confirmar o contrato ANYCAR e completar mapeamentos específicos; habilitar meios de pagamento oferecidos à conta; definir operação de perda de celular, regras de estorno, retenção e rotina administrativa. Não há painel de administrador nem estorno automático implementado.

Ordem recomendada:

1. Criar Supabase e configurar conexão restrita/TLS.

2. Homologar e-mail, Google e MFA.

3. Validar o contrato ANYCAR e suas respostas.

4. Configurar Mercado Pago e suporte.

5. Publicar homologação HTTPS e executar os testes reais.

6. Validar monitoração, backup, cron e rollback.

7. Ativar ANYCAR_CONTRACT_VERIFIED e CHECKOUT_ENABLED.

8. Só então disponibilizar as vendas.

Os nomes e capacidades dos planos de hospedagem podem mudar. As referências oficiais foram consultadas em 11/11/2026. Confirme elegibilidade e limites no momento da contratação; o projeto foi estruturado para mudar de Vercel para um processo Node.js sem mover o banco.